

A Policy–Coverage Duality for Fungal PKS

Reducing genome-to-structure prediction to operator soundness: the duality (proven), the floor (characterized), the loop (sound, not shown convergent), the open step (the map’s hard families)

Brage Eilertsen

brageei@uio.no University of Oslo

Working note, May 2026 (Paper III seed — to be hardened)

Abstract

The design paper recast fungal polyketide prediction as inference over latent programs $y = \text{Exec}_\Theta(z; G)$, with an identifiability–controllability duality: prediction ambiguity and design freedom are one quantity κ_O (Theorem 1, Eilertsen 2026a). The map paper gave a coverage census and a sound-expressibility criterion — an operator is buildable iff its formula-invariant DOF is a computable function of recoverable $\varphi(B)$ — and found that 86% of in-scope cores touch a family whose sound expressibility is *unknown* (Eilertsen 2026b). **This note does not solve fungal PKS and does not claim a near-solution.** It *reduces* the problem and characterizes what a solution would require. Proven: (i) κ_O **factorizes** (Theorem A) into an exact additive ledger over typed gates, so hardness is per-gate and the training loss is counted in gates, not clusters; (ii) **expressibility and learnability are dual** (Theorem B, the central contribution) — learning the per-cycle policy (cycle scale) and extending the operator grammar (operator scale) are one predicate, $d(b) \in \text{image}(\varphi)$, both bottoming out at Rung 4 (their finer relation is contingent, Rem. 3); (iii) the wake–sleep loop built on these is **unconditionally sound** (Proposition 1). The honest negative space, stated as such: the loop’s coverage-growth step *is* the map’s open problem, not a subroutine (Lemma 1); $\mathcal{K}$ -descent is *conditional* on the exchange rate λ (Proposition 2); convergence to the floor is *open* and equals the coverage question (Remark 4); and the floor’s narrowness is itself conditional on a structure-to-kinetics chain we cannot here verify. The contribution is therefore a reduction (solve-fungal-PKS $\Rightarrow$ supply the rungs of the soundness-open families), the policy/coverage duality, a *characterized* (not attained) floor, and four falsifiable predictions — chiefly P1, a Gate-3-specific accuracy lift, absent at Gate-2, whose non-differential appearance would falsify the rung assignment.

1 The one-function reduction (inherited, sharpened)

A program is a cycle list $z = (c_t, a_t)_{t=1}^N$ rendered left-to-right by a sound deterministic executor $\text{Exec}_\Theta(\cdot; G) : \mathcal{Z}(G) \rightarrow \mathcal{Y}$; an observable set O projects structures to signatures $\text{obs}_O : \mathcal{Y} \rightarrow \mathcal{S}$, and $\varphi_O = \text{obs}_O \circ \text{Exec}_\Theta$ is the observable map of a program. The verifier returns the consistent set $Z^*(z; O) = \{z' : \varphi_O(z') = \varphi_O(z)\} = [z]_O$, with $\kappa_O(z) = |[z]_O|$ (Theorem 1, 2026a).

The outlook of 2026a frames the iterative synthase as a partially observed deterministic control system with four objects: the transition $s_{t+1} = F(s_t, a_t)$, the emission $y = \text{readout}(s_N)$, the posterior Z^* , and the per-cycle policy $\pi(a \mid s_t, \varphi(G))$. *Three are known exactly*: F is the executor (applying an operator to the ACP-tethered intermediate *is* F), the emission is the readout, Z^* is the verifier. Direct BGC $\rightarrow$ structure regression stalled at 51–68% because it learned the whole composition end-to-end from terminal data; the executor returns three factors and leaves one. We take this

as the starting point — with a caveat the map paper forces and that this note keeps central: the reduction holds *only over a fixed grammar*. “Solving fungal PKS” is system identification of a single policy π *plus* extension of the grammar to cover the deposited manifold, and the second half is the map’s open problem (§4, §8). The rest of this note makes both precise and is careful to prove only what survives *around* the open step, not the step itself.

2 Theorem A: the hardness factorizes over typed gates

Because Exec_Θ renders programs left-to-right and is deterministic, the consistent set is prefix-closed: $Z^*(y; O)$ is exactly the leaf set of a finite rooted tree $T(y; O)$ — the *program trie* — whose depth- t edges are the per-cycle actions a_t retained after O -pruning (this is the prefix trie of §15, 2026a, and its branch points are the Gates of the Outlook). Let $\mathcal{B}(y; O)$ be the internal nodes of T with ≥ 2 children (the *gates*), and for a gate b let $\deg(b)$ be its number of surviving children.

Theorem A (κ_O -factorization). *For every product y reachable under (G, Θ) and observable set O ,*

$$\kappa_O(y) = 1 + \sum_{b \in \mathcal{B}(y; O)} (\deg(b) - 1). \quad (1)$$

Consequently $\kappa_O(y) = 1$ (the product is exactly identified, and by Theorem 1 exactly specified for design) iff $T(y; O)$ has no gates: every cycle’s action is forced by the prefix and the observables.

Proof. $Z^* = [z]_O$ equals the leaf set of T by prefix-closedness and the verifier’s soundness and relative completeness (§4.1, 2026a): every returned program executes to a structure $\simeq y$ and every legal such program is returned, so the retained edges are exactly the O -consistent continuations. For any finite rooted tree the number of leaves equals $1 + \sum_v (\deg(v) - 1)$ summed over internal nodes, and internal nodes with a single child contribute 0; restricting the sum to multi-child nodes gives (1). The $\kappa_O = 1$ characterization is immediate. $\square$

What is new. Theorem 1 (2026a) gives κ_O as a cardinality; Theorem A makes it an *exact additive ledger over typed gates*. Each gate inherits the Outlook’s type: a *Gate-2* (candidates differ in a per-cycle reduction label, resolvable by a running substrate count — the Substrate-Prefix Engine of §11) or a *Gate-3* (candidates share a reduction trajectory and differ only in the mass-neutral position of a C-methylation — label-symmetric, requiring the Geometric-Rescue Engine). Equation (1) says the field’s hardness at y is literally the sum of its gates, and “resolving y ” is closing them one by one. The empirical shape measured in §15 of 2026a — Gate-3 absent over non-methylating aromatics, single-digit within HR, concentrated in small methylating clusters — is now the *statement that the Gate-3 terms of (1) are sparse and localized*, i.e. the residual sum is short.

Remark 1 (the ledger is the loss). The marginal-likelihood objective $\mathcal{L}(\theta) = -\sum_B \log \sum_{z \in Z^*(B)} \pi_\theta(z | B)$ factorizes over the common prefix of Z^* (§15, 2026a); under (1) it factorizes over exactly the gates of T . A clean trunk edge is a *free, correct* per-cycle label manufactured by the executor; only gate edges carry a learning signal. Training data is therefore counted in *gates*, not clusters — the quantitative engine of §5.

3 Theorem B: expressibility and learnability are one criterion at two scales

A gate b has a *branch-selecting DOF* $d(b)$: which surviving child is the true continuation. Closing b means computing $d(b)$ from recoverable features. The map paper’s criterion (§6, 2026b) — an

operator O is soundly expressible from φ iff its formula-invariant DOF is a computable function of $\varphi(B)$ — is the *same predicate* applied to $d(b)$, and the map paper’s cost ladder classifies *which coordinate of φ supplies it*:

Rung 1 $d(b) = f(s_t)$: the substrate prefix alone (label-distinguishable Gate-2; *free*).

Rung 2 $d(b) = f(s_t, \text{seq})$: a sequence-keyed rule (ESM head; per-module bacterial sequence; PT-clade regiochemistry).

Rung 3 $d(b) = f(s_t, \text{fold/dynamics})$ but not sequence directly: recoverable through $\text{AF2} : \text{seq} \rightarrow \text{fold}$ and $\text{MD/QM} : \text{fold} \rightarrow \text{kinetics}$ (label-symmetric Gate-3; *costly, but a function*).

Rung 4 $d(b)$ is not a function of any cluster feature: an intrinsic mixture (a within-enzyme collision: one sequence, one substrate, ≥ 2 products) or an extra-genomic determinant. *No function exists.*

A gate may be *in-grammar* (its true child is present in T , awaiting selection — the policy’s job) or a *boundary gate* (the true child is absent because Θ cannot express the operator — coverage’s job: admit the branch, *then* select). This is the only structural difference between the two scales.

Theorem B (expressibility–learnability duality). *Fix (G, Θ, O) . Define the residual-DOF set*

$$\Delta(G, \Theta, O) = \{d(b) : b \text{ a gate, in-grammar or boundary, with } \text{rung}(b) \text{ not yet supplied}\}.$$

Write $\Delta_\varphi^{\text{pol}}$ and $\Delta_\varphi^{\text{cov}}$ for the in-grammar gates the policy cannot resolve and the boundary operators coverage cannot express at budget φ ; both are readouts of the one predicate $d(b) \in \text{image}(\varphi)$ — closing a gate (policy) and expressing an operator (coverage) alike. Over the information operator $I_\varphi(b) = \mathbf{1}[d(b) \in \text{image}(\varphi)]$ both residuals shrink monotonically as φ climbs, since $\text{image}(\varphi)$ only grows; the finite-sample empirical error need not (Rem. 2). Invariant (proven): the limits $\Delta_\infty^{\text{pol}}$ and $\Delta_\infty^{\text{cov}}$ consist entirely of Rung-4 obstructions — $d(b)$ a function of no cluster feature — which no feature budget removes. Topology (contingent): their relation — shared core, containment, or incomparability — is fixed by the executor’s representation of non-functional steps (Rem. 3), a design choice over case populations not yet instantiated in the corpus.

Proof sketch. By Theorem A the policy’s only learnable targets are the in-grammar gates; its irreducible error is the entropy of those $d(b)$ not determined by φ , zero exactly when φ contains each gate’s rung coordinate (the criterion of §6, 2026b, read at cycle scale). Operator coverage adds a boundary gate’s true child iff that operator’s formula-invariant DOF is a function of φ — the same criterion read at operator scale. Both predicates are “ $d(b) \in \text{image}(\varphi)$ ”, monotone under enlarging φ over the information operator I_φ (a larger feature set cannot make a previously-expressible DOF inexpressible, since the policy may ignore coordinates; the finite-sample error is a separate matter, Rem. 2). Rung 4 is precisely the case where $d(b)$ is not a function of *any* cluster feature — an intrinsic distribution has no point target, an extra-genomic determinant is outside φ by definition — so it lies in neither image at any budget. The invariant is all the proof delivers; the relation between the two limit sets is settled below, not here. $\square$

Remark 2 (information operator vs. empirical error). Theorem B’s monotonicity is a statement about I_φ — whether $d(b)$ is closable *in principle* at budget φ . The quantity P1/P2 measure is the *empirical* held-out error, which decomposes as an approximation term (monotone non-increasing in $\text{image}(\varphi)$) plus an estimation term that can *rise* with feature dimension. Adding the high-dimensional AF2 pose trajectory can therefore increase held-out error on a gate a leaner φ already closed, with the information still present. P1 is thus a claim about the approximation term, to be read with capacity control or in the data-rich regime; a variance-driven bump is not a falsification.

Remark 3 (the Rung-4 floor: four populations, presently uninstantiated). Enumerating what can be Rung-4 gives four populations. $A(i)$: a genuine mixture with both products buildable — the verifier returns $\kappa_O \geq 2$ and no sound function exists, so it sits on *both* floors (the shared core). $A(ii)$: extra-genomic routing among buildable children — irreducible κ_O but no missing operator, hence *policy floor only*. B : a Rung-4 operator whose true child is unbuildable while its gate still surfaces in-grammar via other children — *coverage floor only* (Lemma 1). C : a Rung-4 operator occurring only inside otherwise-out-of-grammar programs, never surfacing in-grammar — *coverage floor only*. Thus $\Delta_\infty^{\text{pol}} = \{A(i), A(ii)\}$ and $\Delta_\infty^{\text{cov}} = \{A(i), B, C\}$: incomparable in general; containment $\Delta_\infty^{\text{pol}} \subseteq \Delta_\infty^{\text{cov}}$ iff $A(ii) = \emptyset$; equality iff also $B = C = \emptyset$; set-identity (the discarded “identical” claim) iff both wings vanish. Two construction facts decide which holds. **Q1**: can a Rung-4 gate occur only in out-of-grammar programs (C)? **Q2**: does the executor represent a non-functional step as one in-grammar gate with $\kappa_O \geq 2$ (policy side) or exclude it by a soundness-requires-a-function rule (coverage side), and is Δ^{cov} indexed by missing buildable children or by missing sound functions? Both are *design decisions over case populations not yet instantiated*: the corpus holds no confirmed Rung-4 core, so $A(ii), B, C$ are presently empty and even $A(i)$ is unconfirmed (zero verified within-enzyme collisions). The invariant is proven; the topology is a statement about possibly-vacuous sets, resolved by construction choices not yet pinned — and if no $A(i)$ is ever confirmed the floor is itself empty, which is its own finding: Rung 4 conjectured, never demonstrated.

This is the companion to Theorem 1 (2026a). Theorem 1: *predict* $\leftrightarrow$ *design* are one κ_O . Theorem B: *learn-policy* $\leftrightarrow$ *extend-grammar* share one predicate $d(b) \in \text{image}(\varphi)$ and a Rung-4 floor-character, with two (in general incomparable) residuals (Rem. 3). And the two dualities are the *same statement at two scales* of the program grammar, summarized in Table 1: one predicate, “is the hidden DOF a computable function of recoverable φ ?”, governs all four corners.

	inference question	action question
cycle scale (a gate $d(b)$)	predict the per-cycle action ↓ one quantity κ_O (Thm. 1) ↓	specify/edit the iteration program ↓ one quantity κ_O (Thm. 1) ↓
operator scale (an operator DOF)	is this product reconstructible? ↓ one set Δ (Thm. B) ↓	is this operator buildable? ↓ one set Δ (Thm. B) ↓
closes by	climbing $\varphi = \text{seq} \rightarrow \text{fold} \rightarrow \text{dynamics}$; floor at Rung 4	

Table 1: The master 2×2 . A single predicate — $d \in \text{image}(\varphi)$ — governs prediction, design, policy learning, and operator coverage. Theorem 1 unifies the top row’s columns; Theorem B unifies the rows; the rung ladder supplies the common closing mechanism.

4 Lemma 1: coverage strictly precedes policy

Lemma 1 (coverage-gates-policy). *If the in-vivo program z^* of a target y is out-of-grammar, $z^* \notin \mathcal{Z}(G)$, then $z^* \notin Z^*(y; O)$ for every O , so no policy π ranking within Z^* can return z^* . Hence on out-of-grammar cores the marginal value of any policy is zero, and operator extension is a strict prerequisite.*

Proof. $Z^*(y; O) \subseteq \mathcal{Z}(G)$ by construction; $z^* \notin \mathcal{Z}(G)$ gives $z^* \notin Z^*$. A policy is a reweighting of Z^* , whose support excludes z^* . $\square$

Citrinin is the worked instance. In §11 (2026a) the geometric-rescue head drove held-out entropy to 0.01 and committed $\sim 99.9\%$ mass to a single in-grammar candidate — yet the true biology requires two C-methylation events, an *out-of-grammar* program at that formula. The

entropy collapse proves the structural channel carries discriminating information (the policy is real); Lemma 1 explains why it nonetheless selected the wrong subtree (the truth was off the trie). The moral is an *ordering* on the loop of §5: spend on coverage until a core’s true program is in-grammar; only then does policy resolution of its gates mean anything. Confident closure of an in-grammar gate on an under-covered core is the failure mode to guard against, and it is detectable — a verified verdict that the metabologenomics triage of §9 (2026a) would route to expression+NMR.

5 The loop: a sound, self-funding scaffold around an open step

Theorems A–B and Lemma 1 assemble into one procedure (Figure 1), a literal wake–sleep instantiation of the execution-guided program synthesis the design paper cites (DreamCoder; Ellis et al. 2021):

1. **Sleep (dream).** The sound executor, run forward as a generator, enumerates producible programs and renders them to structures — an unbounded synthetic $(\text{structure}, \varphi) \rightarrow z$ corpus. Pretrain the shared policy π_θ to invert it. This amortizes the verifier’s enumeration into a network and shapes π by the chemistry before any real label is seen.
2. **Wake (verify).** On real clusters the verifier returns Z^* ; by Theorem A its clean trunk supplies free correct per-cycle labels and its gates supply the learning signal. Fine-tune π_θ on these manufactured labels under the factorized marginal-likelihood loss. *One shared π for the whole family:* the compression that hides per-step state from a static reader (the very cause of the 0.567 wall) is what makes all HR-PKS share a single policy, so lovastatin’s and citrinin’s cycles train the same function — the curse read as a transfer blessing.
3. **Abstraction (grow coverage) — this step *is* the open problem, not a subroutine.** Two extensions, both *verifier-gated*: (a) reusable cycle motifs discovered in Z^* become library operators (sound by construction); (b) for each hard family the Rung 0–4 instrument (§6, 2026b) *would* adjudicate a learned, gene-conditioned operator, admitted only if it passes the mass/formula gate *and* a held-out $\simeq$ -match. But whether (b) *succeeds* is precisely the map’s unsolved question: the instrument has returned exactly one verdict (aromatic cyclization, Rung 2, on closures curated and encoded before the instrument existed), while macrolactonization (Rungs 2–3, unmeasured) and oxidation/rearrangement (unassessed) together touch 86% of in-scope cores. So coverage growth past the eight reachable cores is *conditional on rung assignments that are open*; by Theorem B’s own logic, if those families are Rung 4 the loop provably cannot reach them. The loop is well-defined and sound (Prop. 1) over families whose rung is established; over the rest it *names* the work, it does not perform it. New operators, where they exist, admit boundary gates’ true children (Lemma 1), enlarging $\mathcal{M}(G)$.
4. **Direct (the planner is already the controller).** The experiment-selection planner of §9.5 (2026a) — by Corollary 1 there already a design planner — is *also* the loop’s controller: it ranks, by expected $\Delta\kappa_O$ per unit cost, both which folds/structures to compute (which φ -coordinate to buy for which Gate-3 cluster) and which operators to encode next. Because the Gate-3 ledger of Theorem A is sparse and localized, the structural-data campaign is small and targeted, not a structural-genomics sweep.

Three claims about the loop must be separated, because they have different status; conflating them is how a sound scaffold gets mistaken for a solution.

Proposition 1 (monotone soundness — proven, unconditional). *Every iterate of the loop emits only mass/chemistry-consistent, $\simeq$ -certified structures. Each learned component is verifier-gated, so its outputs lie in Z^* intersected with verifier-passing proposals, a subset of the sound set (§4.1, 2026a). Soundness is therefore invariant along the loop, with no condition on φ , on λ , or on the coverage question.*

Proposition 2 ($\mathcal{K}$ -descent — conditional on λ). *Under policy improvement at fixed membership the in-grammar ledger of (3) is non-increasing (per-core κ_O is a min over a superset of decision rules as φ climbs), and coverage is append-only. But admitting a previously out-of-grammar core y moves it from the coverage-gap term into the ledger, changing $\mathcal{K}$ by*

$$\Delta\mathcal{K}(y) = \log \kappa_O(y) - \lambda c(y), \quad (2)$$

unsigned in general: a newly admitted hard core arrives carrying its own gates, so $\log \kappa_O(y)$ may be large. Hence $\mathcal{K}$ is non-increasing along the loop iff $\lambda c(y) \geq \log \kappa_O(y)$ for every admitted core — a condition on λ , not a theorem. Forcing it with large λ has a cost: $\mathcal{K}$ becomes dominated by the coverage-gap count, so “ $\mathcal{K}$ small” ceases to mean “predictions are good” and means only “few cores remain out-of-grammar.” The earlier draft asserted unconditional monotonicity; (2) is the term that assertion omitted, and §8(2) is its counterexample.

Remark 4 (convergence to the floor — open). Even where Proposition 2’s condition holds, monotone descent of a bounded-below $\mathcal{K}$ gives convergence to *some* limit, not to $\mathcal{K}_\infty$. Attaining the floor additionally requires that descent not stall above it — that every above-floor gate is eventually closed and every in-scope family eventually covered at finite cost. That is exactly the map’s open coverage question (§4, §8), not a consequence of the loop’s structure. Theorem B characterizes *where* the floor is (Δ_∞ , Rung 4); nothing here proves the loop *reaches* it. The gap between “the floor is X ” and “the procedure attains X ” is the gap between this note and a solution.

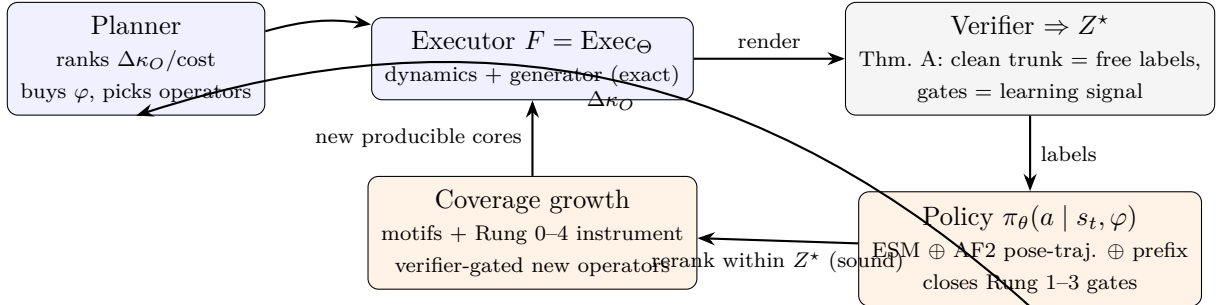


Figure 1: The closure loop. Blue = symbolic and exact (executor, planner); orange = learned but verifier-gated (policy, coverage). The executor manufactures the policy’s supervision (Thm. A); the policy and coverage growth shrink the residual set Δ (Thm. B) along $\varphi = \text{seq} \rightarrow \text{fold} \rightarrow \text{dynamics}$; the planner directs spend by expected $\Delta\kappa_O$ per unit cost; soundness is invariant (Prop. 1), while $\mathcal{K}$ -descent is only conditional (Prop. 2) and convergence is open (Rem. 4). Coverage precedes policy on any out-of-grammar core, and step (3b) is itself the open problem (Lemma 1).

6 $\mathcal{K}$: a measurable definition of done (target characterized, attainment open)

Theorems A and B let us replace the vibe “is fungal PKS solved?” with a scalar. Over the deposited, study-biased manifold $\mathcal{D}$ (the 326 MIBiG fungal PKS), define the *residual ambiguity*

$$\mathcal{K}(G, \Theta, \varphi; O) = \underbrace{\sum_{y \in \mathcal{D} \cap \mathcal{M}(G)} \log \kappa_O(y)}_{\text{in-grammar gate ledger (Thm. A)}} + \lambda \underbrace{\sum_{y \in \mathcal{D} \setminus \mathcal{M}(G)} c(y)}_{\text{coverage gap (Lemma 1)}}, \quad (3)$$

with $c(y)$ the planner’s cost to admit y ’s operator and $\lambda > 0$ a fixed exchange rate. By Proposition 1 the loop is sound unconditionally; by Proposition 2 $\mathcal{K}$ descends only under the stated λ -condition; and by Remark 4 convergence to the floor is *not* established here — it is the coverage question. Theorem B identifies *where* the floor sits,

$$\mathcal{K}_\infty = \sum_{y: y \text{ carries a Rung-4 gate}} \log(\text{irreducible multiplicity of } y),$$

a quantity we can *estimate now* by tagging within-enzyme collisions (one sequence, one substrate, ≥ 2 products) and trans-/extra-genomic determinants across $\mathcal{D}$. “Fungal PKS is solved” then *means* precisely $\mathcal{K} \rightarrow \mathcal{K}_\infty$: every product either exactly programmed ($\kappa_O = 1$) or certified Rung-4-irreducible. This defines done; whether the loop can drive $\mathcal{K}$ there is the open content of Remark 4, not a claim of this note. The move — turning a hardness claim into an exact object over the executor’s certifiable reach — is the same one both prior papers make, now applied to the *definition* of done rather than to its attainment.

7 Falsifiable predictions and the experiment that starts it

The program is testable on assets the design paper already built.

- P1 (Gate-3-specific lift — the first measurable breakthrough).** Inject the conformational coordinate of s_t as a *pose trajectory*: 3D-embed the ACP-tethered intermediate at *each* cycle’s chain length (the per-cycle geometry §11 already computes) rather than one static pocket. Prediction: held-out per-gate accuracy rises above the Rung- ≤ 2 ceiling *only at Gate-3 (label-symmetric) gates* and is unchanged at Gate-2 gates (Rung 1 already closes those). A non-differential, across-the-board lift would falsify the rung assignment. Citrinin ($H = \log 2 \rightarrow 0.01$, §11) is the existence proof for one gate; P1 is that it generalizes per-gate, not per-cluster.
- P2 ($\mathcal{K}$ behaviour, with the λ -caveat).** Steps (i) adding the ESM head (closes Rung-2 gates, chiefly bacterial) and (ii) adding the pose trajectory (closes Gate-3) are pure policy improvement and lower $\mathcal{K}$ *unconditionally*. An admission step (iii) can *raise* $\mathcal{K}$ by $\log \kappa_O(y) - \lambda c(y)$ (Eq. (2)); observing the sign per family is itself a measurement of whether the hard cores arrive gate-heavy. A claim of unconditional descent across (i)–(iii) is the one this note retracts; the falsifiable content is that (i)–(ii) descend while (iii) is signed by (2).
- P3 (coverage-gates step).** On a core whose true program is out-of-grammar (e.g. citrinin’s two-C-MeT program at its formula), recovery is flat under *every* φ improvement and steps up only when the operator is encoded — a flat-then-step curve. A smooth climb would falsify Lemma 1.
- P4 (RiPP transfer, from Corollary 3, 2026a).** Instantiate a RiPP grammar in the shared interface. Prediction: κ_O is small on core-residue gates (high identifiability, easy design) and

large on modification-topology gates, which sit at Rung 3–4; the same loop closes the former trivially and stalls on the latter at the identical rung structure — the duality surviving a family the engine has not implemented.

The buildable architecture (today). A shared policy head over the legal continuations the verifier returns, with features

$$\varphi(G, s_t) = \underbrace{\text{ESM-2(per-domain seq)}}_{\text{Rung 2, where seq varies}} \oplus \underbrace{\text{AF2-pose}(s_t)}_{\text{Rung 3, per-cycle geometry}} \oplus \underbrace{\text{substrate-prefix counters}}_{\text{Rung 1}}$$

trained on Theorem-A-manufactured labels under the factorized marginal-likelihood loss, jointly across the producible manifold. This is §11 of 2026a scaled from three assets to the manifold, planner-directed at the sparse Gate-3 ledger — the concrete first step, not a new apparatus.

8 What this does not claim (the honest floor)

The program is bounded by the same discipline as its parent papers, and four limits are structural, not provisional. **(1) Coverage is prior and harder.** The soundness-open families (macrolactonization in 176/206 in-scope cores; §4, 2026b) gate everything downstream by Lemma 1; if a substantial fraction is soundly inexpressible, complete genome-to-product prediction is permanently partial — a finding, not a gap. **(2) The two frontiers are coupled.** The Gate-3 ledger of Theorem A is measured only over currently-reachable cores; as coverage grows, the pocket-hardest chemistries (displaced past the coverage boundary today — citrinin is the tell) may surface as new Gate-3 terms, so the residual sum can inflate before it shrinks. **(3) Rung 4 is a real floor in principle — but presently a conjectural one.** Where an enzyme makes a genuine mixture or the determinant is extra-genomic, there is no function to learn and the obstruction is Rung-4; the definition of done is $\mathcal{K} \rightarrow \mathcal{K}_\infty$, never $\mathcal{K} \rightarrow 0$. But no such case is yet confirmed in the corpus (Rem. 3), so $\mathcal{K}_\infty$ is at present *conjectural* — possibly non-empty, possibly vacuous — and the latter would itself be a finding (Rung 4 defined, never demonstrated). The floor’s existence, like its narrowness (4), is inherited as a conditional claim, not a proven one. **(4) The floor’s narrowness is itself conditional.** The argument that Rung 4 is “high and narrow” — that dynamic/kinetic selection is a costly Rung 3 rather than inexpressible — leans on the chain $\text{AF2} : \text{seq} \rightarrow \text{fold}$ and $\text{MD/QM} : \text{fold} \rightarrow \text{kinetics}$ being computable *in practice*, not merely in principle. If that chain is not practically attainable, Rung-3 cases leak toward the floor, Δ_∞ effectively widens, and “expressible almost everywhere” softens to “expressible where the structure-to-kinetics chain is tractable, unknown elsewhere.” We cannot settle this here; we flag the floor’s narrowness as a conditional claim inherited from 2026a/b, not a proven one. Within these bounds the contribution stands as what it is: a reduction (solve-fungal-PKS to operator soundness), the policy/coverage duality (Theorem B, the genuine result), an unconditionally sound scaffold (Prop. 1), and a *characterized* floor — not a closure. The loop’s descent is conditional (Prop. 2) and its convergence open (Rem. 4); the irreducible residual is a set we can name and estimate but have *not* shown the procedure attains. That is a program worth pursuing, stated as a reduction-plus-duality rather than a solver with an accuracy — and the honest object the field can use today remains the map.

References

- [1] B. Eilertsen. *Sound Biosynthetic Design over Iterative Grammars: A Realizability Geometry for Fungal Polyketides* (2026a). github.com/BrageEilertsen/lpi-fungal.

- [2] B. Eilertsen. *A located map of the fungal polyketide problem* (2026b).
- [3] K. Ellis et al. *DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning*. PLDI, 2021.
- [4] Z. Lin et al. *Evolutionary-scale prediction of atomic-level protein structure with a language model* (ESM-2). Science, 2023.
- [5] R. J. Cox. *Curiouser and curiouser: programming of iterative highly-reducing PKS*. Nat. Prod. Rep., 2023.